

ENSF 380

TOPIC 01: INTRODUCTION TO JAVA

Why Study Principles of Software Engineering?

Builds on past learning of syntax and programming techniques

Understanding fundamentals sets the stage for further study

Principles apply regardless of programming language, specialization

Why Learn Java?

Widely used in industry

Widely used in open source software (OSS)

Key Features

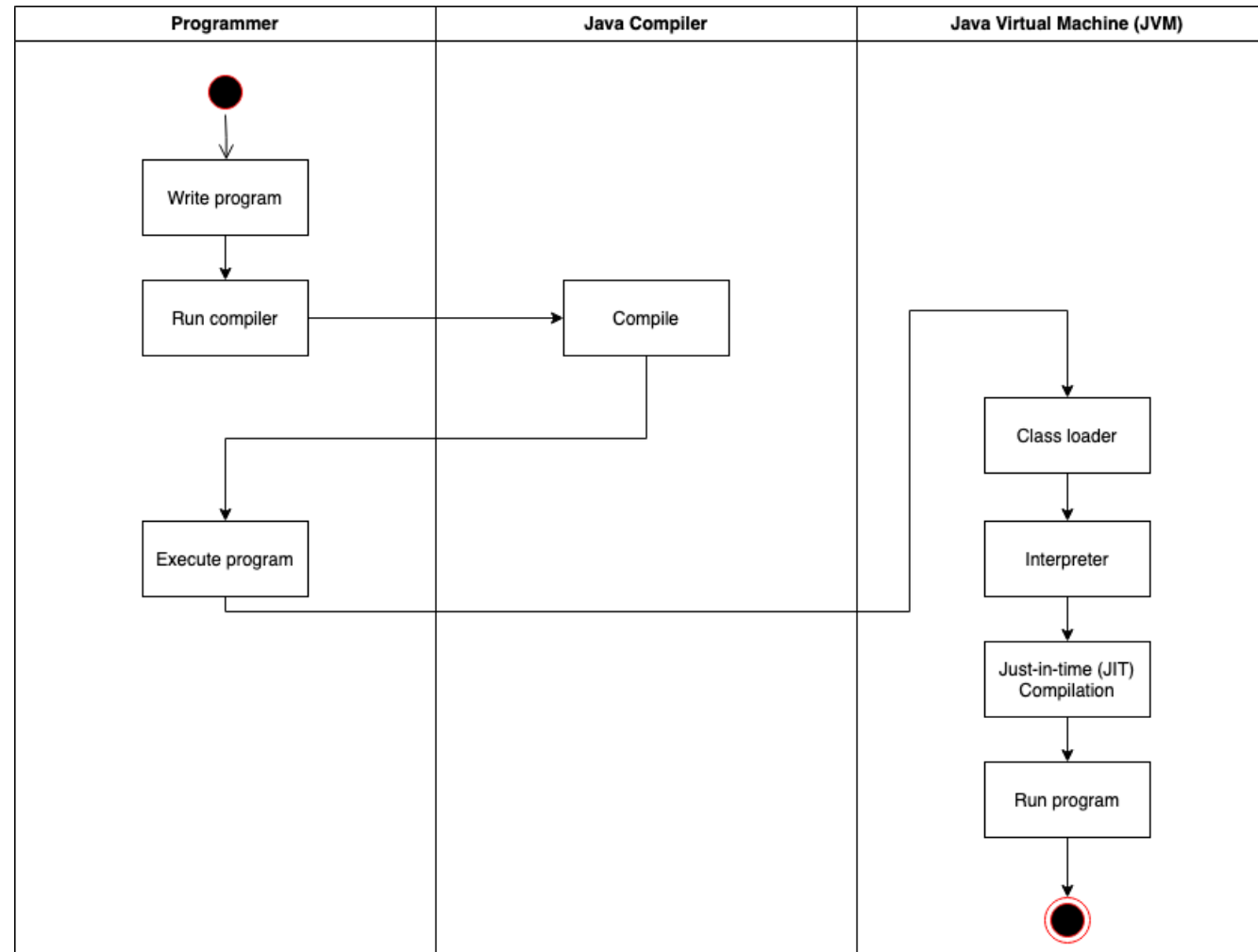
Object-oriented

Distributed

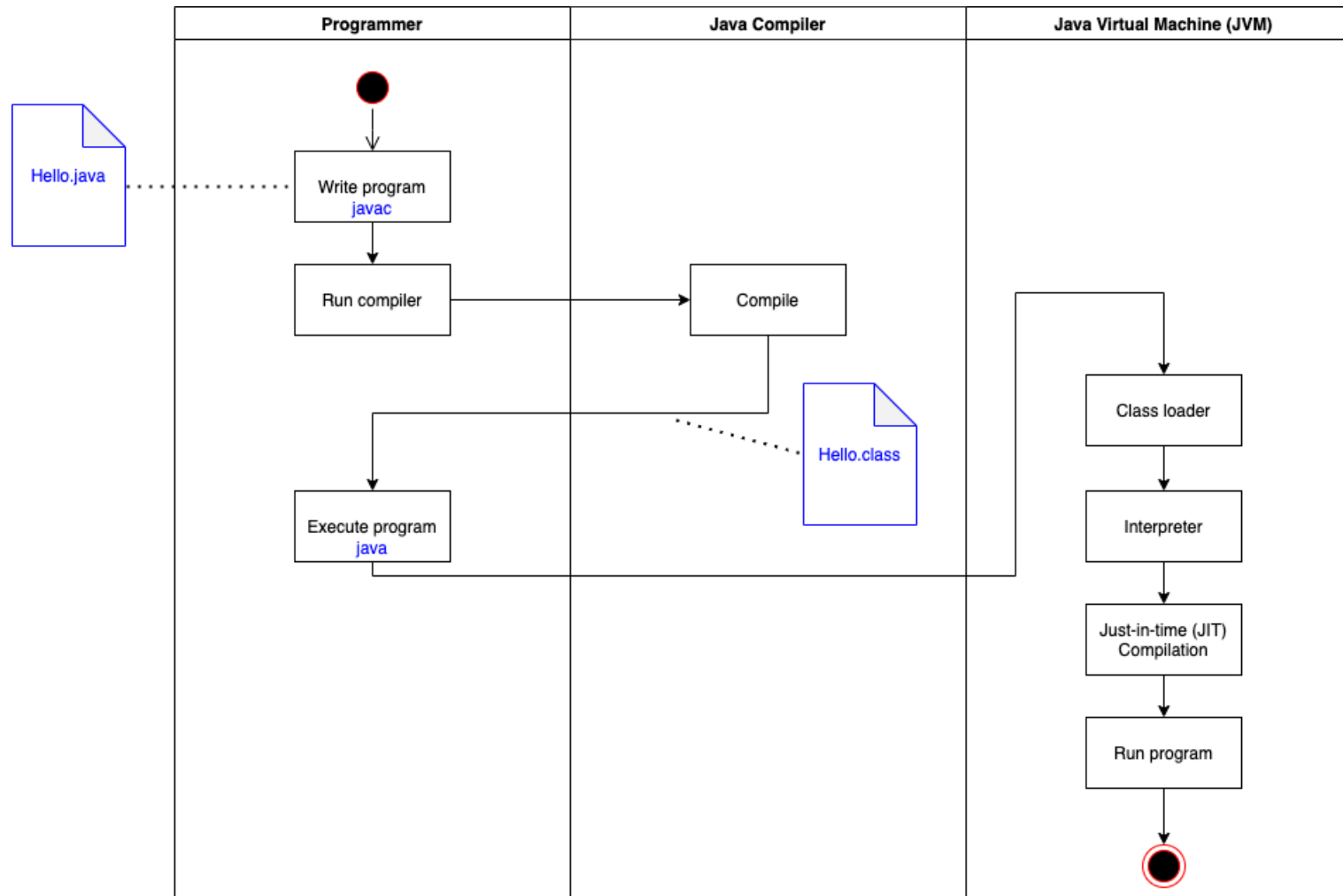
Multithreaded

Interpreted

Writing and Executing a Java Program



Writing and Executing a Java Program



Source Code vs. Byte Code

Hello.java

```
public class Hello {  
    public static void main(String[] args) {  
        System.out.println("Hello, world");  
    }  
}
```

Context box

Hello.class

```
????;  
  
java/lang/Object<init>()V  
  
java/lang/System.outLjava/io/PrintStream;  
                                Hello, world  
  
java/io/PrintStreamprintln(Ljava/lang/String;)VHelloC  
+odeLineNumberTablemain([Ljava/lang/String;)V  
SourceFile  
??llo.java!*??    %    ?
```

Java Source Code vs. C++ Source Code

Hello.java

```
public class Hello {  
    public static void main(String[] args) {  
        System.out.println("Hello, world");  
    }  
}
```

Hello.cpp

```
#include <iostream>  
using namespace std;  
  
int main() {  
    cout << "Hello, world" << endl;  
    return 0;  
}
```


Organization and Importing

Related classes are grouped as packages

May be built-in or user-defined

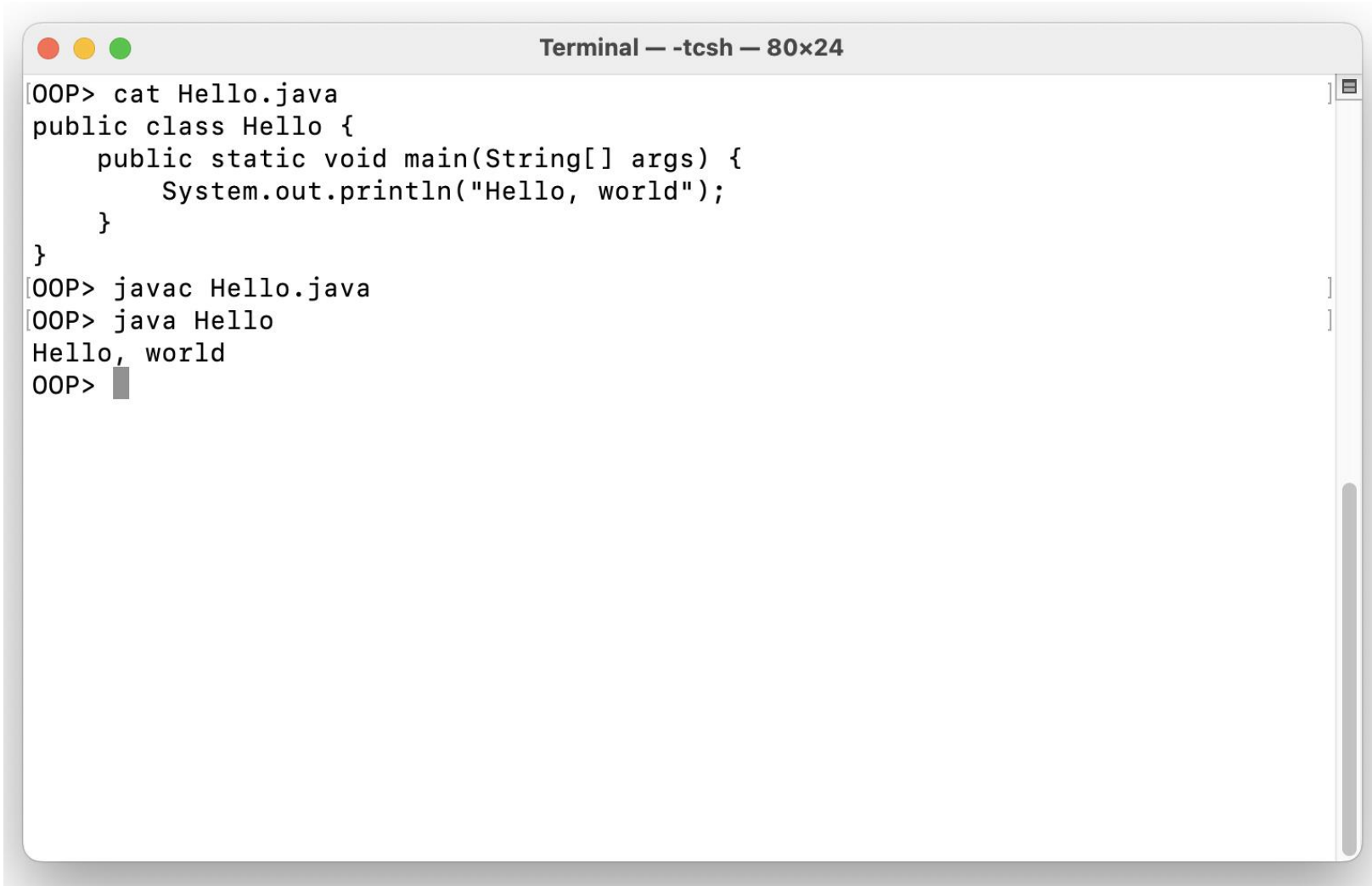
Use “import” statement to include packages or single classes

- e.g. `import java.util.*;`
- e.g. `import java.util.Scanner;`

Exercise 1.1

TEST YOUR UNDERSTANDING BY COMPLETING THE EXERCISE

Writing and Executing a Java Program

A screenshot of a macOS Terminal window titled "Terminal — -tcsh — 80x24". The window contains the following text:

```
[OOP> cat Hello.java
public class Hello {
    public static void main(String[] args) {
        System.out.println("Hello, world");
    }
}
[OOP> javac Hello.java
[OOP> java Hello
Hello, world
[OOP> █
```

The code defines a class named 'Hello' with a 'main' method that prints 'Hello, world'. It then compiles the file with 'javac' and runs it with 'java', resulting in the output 'Hello, world'. The prompt '[OOP>' is used throughout the terminal session.

Commenting a Java Program

Java has two commenting styles

- Same line
- Separate line

```
/*  
Hello is a simple example program which prints the classic message:  
Hello, world  
*/  
public class Hello {  
    public static void main(String[] args) {  
        System.out.println("Hello, world");  
    }  
} // End of class declaration
```

Documenting a Java Program

Comments are for a programmer editing the code

Documentation is for a programmer using the code, who does not need to know implementation details

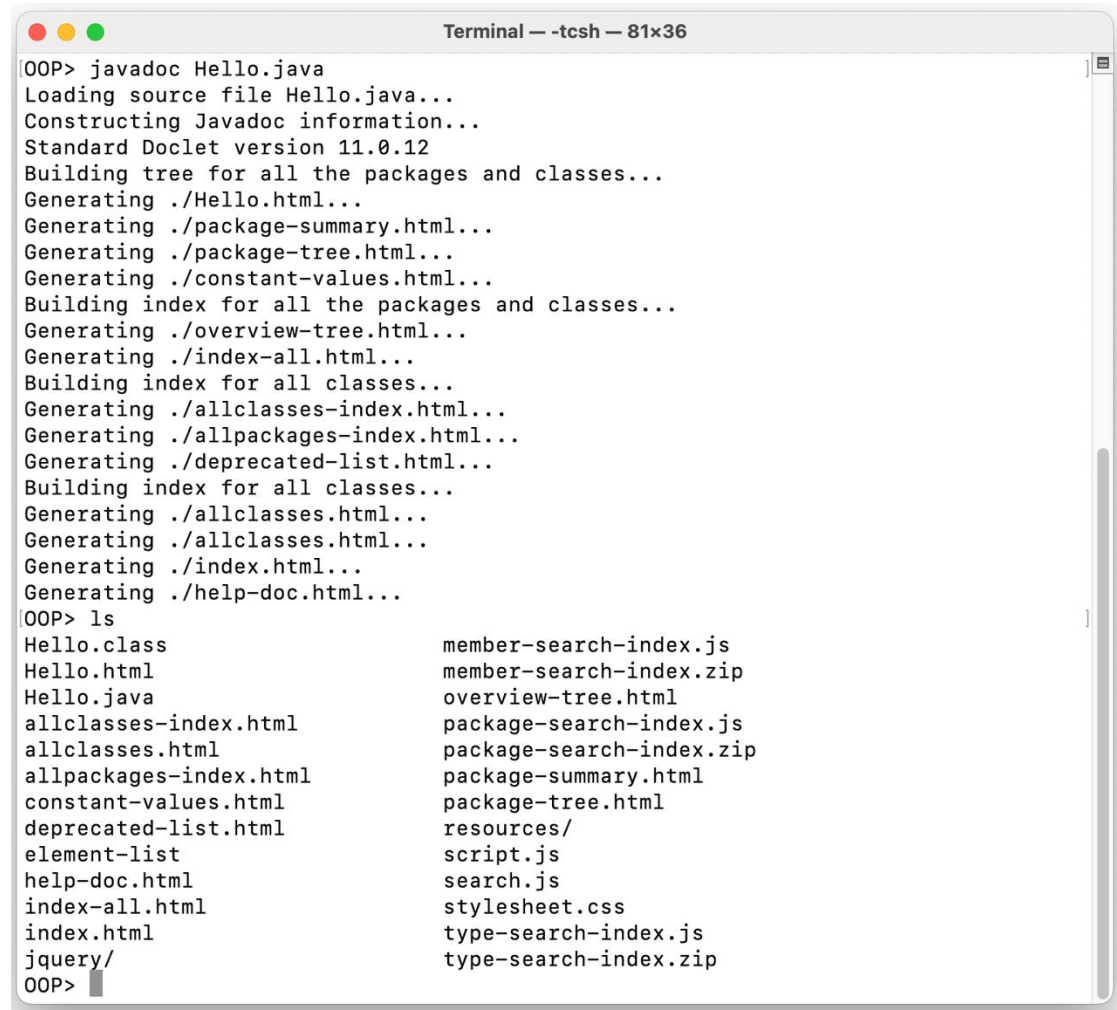
Documentation in Java uses a modification of comment syntax

Documentation can contain HTML

Exercise 1.2

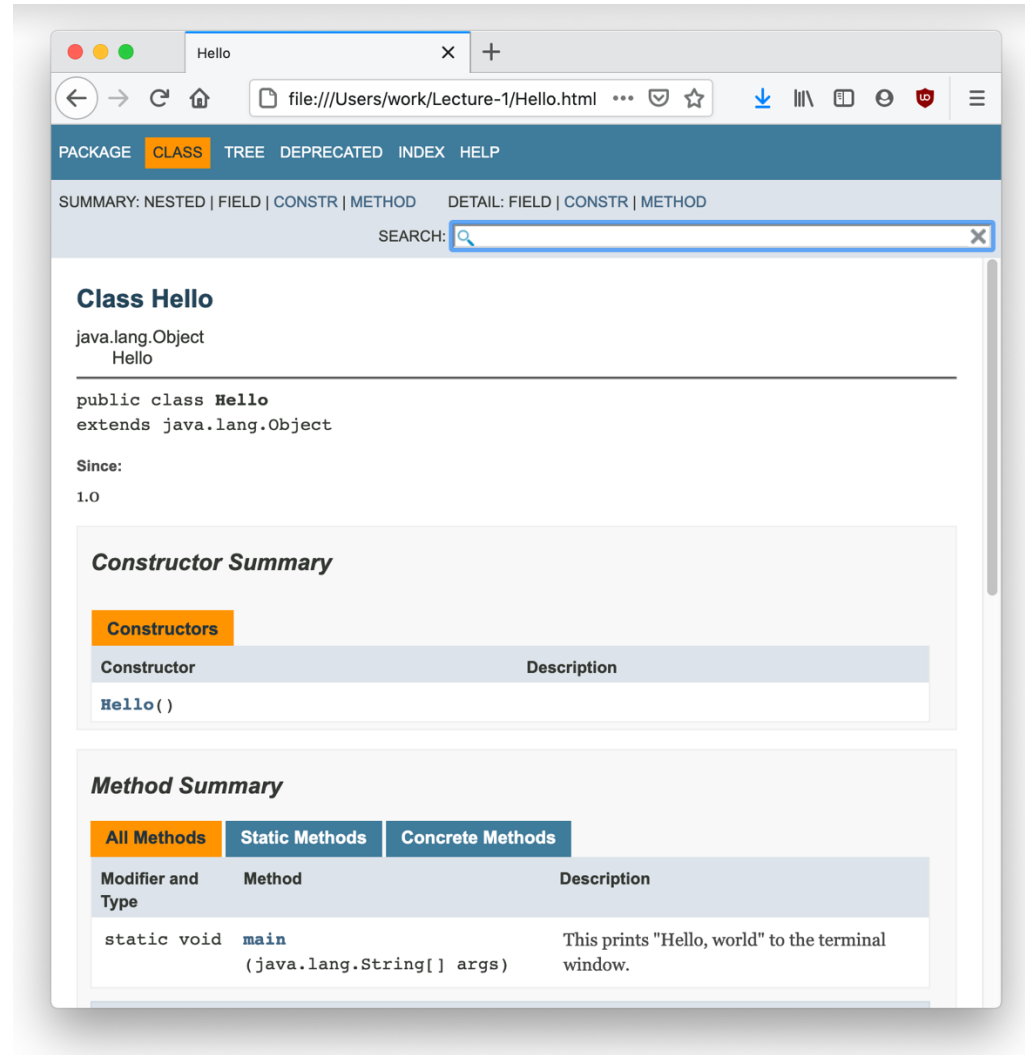
TEST YOUR UNDERSTANDING BY COMPLETING THE EXERCISE

Documenting a Java Program



```
Terminal — -tcsh — 81x36
OOP> javadoc Hello.java
Loading source file Hello.java...
Constructing Javadoc information...
Standard Doclet version 11.0.12
Building tree for all the packages and classes...
Generating ./Hello.html...
Generating ./package-summary.html...
Generating ./package-tree.html...
Generating ./constant-values.html...
Building index for all the packages and classes...
Generating ./overview-tree.html...
Generating ./index-all.html...
Building index for all classes...
Generating ./allclasses-index.html...
Generating ./allpackages-index.html...
Generating ./deprecated-list.html...
Building index for all classes...
Generating ./allclasses.html...
Generating ./allclasses.html...
Generating ./index.html...
Generating ./help-doc.html...
OOP> ls
Hello.class                member-search-index.js
Hello.html                 member-search-index.zip
Hello.java                 overview-tree.html
allclasses-index.html      package-search-index.js
allclasses.html            package-search-index.zip
allpackages-index.html     package-summary.html
constant-values.html       package-tree.html
deprecated-list.html       resources/
element-list               script.js
help-doc.html              search.js
index-all.html            stylesheet.css
index.html                 type-search-index.js
jquery/                    type-search-index.zip
OOP>
```

Documenting a Java Program



The screenshot shows a web browser window with the title 'Hello'. The address bar displays the file path: `file:///Users/work/Lecture-1/Hello.html`. The browser's navigation bar includes buttons for back, forward, refresh, and home, along with a search icon and a star icon. Below the navigation bar, there is a tabbed interface with 'PACKAGE', 'CLASS', 'TREE', 'DEPRECATED', 'INDEX', and 'HELP'. The 'CLASS' tab is selected. Below the tabs, there is a search bar with the text 'SEARCH:' and a magnifying glass icon. The main content area displays the documentation for the 'Class Hello'. It shows the package 'java.lang.Object' and the class 'Hello'. The class is defined as `public class Hello extends java.lang.Object`. Below this, it says 'Since: 1.0'. There are two sections: 'Constructor Summary' and 'Method Summary'. The 'Constructor Summary' section has a sub-tab 'Constructors' and a table with one entry: `Hello()`. The 'Method Summary' section has sub-tabs 'All Methods', 'Static Methods', and 'Concrete Methods'. The 'All Methods' tab is selected, and it shows a table with one entry: `static void main (java.lang.String[] args)` with the description 'This prints "Hello, world" to the terminal window.'

Class Hello

java.lang.Object
Hello

public class **Hello**
extends java.lang.Object

Since:
1.0

Constructor Summary

Constructors

Constructor	Description
<code>Hello()</code>	

Method Summary

All Methods Static Methods Concrete Methods

Modifier and Type	Method	Description
static void	<code>main</code> (java.lang.String[] args)	This prints "Hello, world" to the terminal window.

Primitive Data Types

Type	Contains
boolean	true or false
char	Unicode character
byte	signed integer
short	signed integer
int	signed integer
long	signed integer
float	floating point
double	floating point

Using Primitive Data Types

```

/**
 * @author      Firstname Lastname <a
+ href="mailto:firstname.lastname@ucalgary.ca">firstname.lastname@ucalgary.ca</a>
 * @version     1.3
 * @since      1.0
 */

/*
Hello is a simple example program for lecture 01
*/

public class Hello {
    /**
    This prints "Hello" and the contents of a variable.
    @param args Optional command-line argument
    */
    public static void main(String[] args) {
        int exampleVariable = 100;
        System.out.println("Hello, " + exampleVariable);
    }
} // End of class declaration

```

Using Primitive Data Types

```
+ /**
   * @author      Firstname Lastname <a
   * href="mailto:firstname.lastname@ucalgary.ca">firstname.lastname@ucalgary.ca</a>
   * @version     1.4
   * @since      1.0
   */

   /**
   * Hello is a simple example program for lecture 01
   */

   public class Hello {
       /**
       * This prints "Hello" and the contents of a variable.
       * @param args Optional command-line argument
       */
       public static void main(String[] args) {
           boolean exampleVariable = true;
           System.out.println("Hello? " + exampleVariable);
       }
   } // End of class declaration
```

Type Casting

Convert one primitive data type to another

Widening (Automatic): assign a smaller type to a larger type (e.g. int -> double)

- byte -> short -> int -> long -> float -> double

Narrowing (Explicit): assign a larger type to a smaller type (e.g. double -> int)

- double -> float -> long -> int -> short -> byte

Literals can be used to force types (e.g. `x = 0.5f` will be a float, rather than a double)

Type Casting Examples

Widening (Automatic)

```
... public static void main(String [] args) {  
    int intNum = 25;  
    double doubNum = intNum; //automatic cast  
    System.out.println("Integer value: " + intNum);  
    System.out.println("Double value: " + doubNum);  
}  
...
```

Narrowing (Explicit)

```
... public static void main(String [] args) {  
    double doubNum = 25.5;  
    int intNum = (int)doubNum; //explicit casting  
    System.out.println("Integer value: " + intNum);  
    System.out.println("Double value: " + doubNum);  
}  
...
```

Exercise 1.3

TEST YOUR UNDERSTANDING BY COMPLETING THE EXERCISE

Summary: Knowledge

Execution of a Java program

Java primitive data types

Casting of primitive data types

Summary: Skills

Write a simple Java program

Write comments in a Java program

Write documentation in a Java program

Use the Java commands:

- javac
- java
- javadoc

Additional Information

Related reading

- Volume 1, Chapter 1: An Introduction to Java
- Volume 1, Chapter 2: The Java Programming Environment
- Volume 1, Chapter 3: Fundamental Programming Structures in Java

Primitive data types

- <https://docs.oracle.com/javase/specs/jls/se8/html/jls-4.html#jls-4.2.3>

Type	Contains	Bit size	Default value	Value range
boolean	true or false	1	false	true/
char	Unicode chars	16	\u0000	\u0000 to \uffff
byte	signed integer	8	0	-128 to 127
short	signed integer	16	0	-32768 to 32767
int	signed integer	32	0	-2^{31} to $2^{31} - 1$
long	signed integer	64	0	-2^{63} to $2^{63} - 1$
float	floating point	32	0.0	
double	floating point	64	0.0	